Površina Senke

Rad u okviru kursa
Geometrijski algoritmi
Matematički fakultet

Jovan Vukićević
jovanvukicevic@hotmail.com

2. mart 2026.

Sažetak

Problem površine senke predstavlja geometrijski problem sa širokom primenom u industriji i može poslužiti kao efikasan alat za rešavanje različitih optimizacionih problema, uključujući problem umetničke galerije. U ovom radu prvo izlažemo naivni algoritam za rešavanje ovog problema, a zatim i njegovu optimizovanu, efikasniju verziju. Na kraju samog rada prikazujemo rezultate algoritma za različite ulaze i diskutujemo potencijalna poboljšanja i kritične tačke samog algoritma.

Sadržaj

1 Uvod **2**

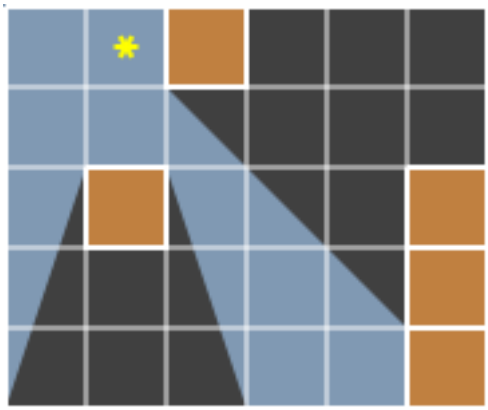
2 Algoritam **2**
2.1 Naivni algoritam 2
2.2 Algoritam brišuće prave 3

3 Implementacija algoritma **3**

4 Vizuelizacija rezultata **5**

1 Uvod

Problem površine senke odnosi se na izračunavanje površine senki koje stvara tačkasti izvor svetlosti u prostoriji sa preprekama. U ovom radu razmatramo specijalan slučaj ovog problema u kome su prepreke ograničene na jedinične kvadrate, a prostorija je modelovana kao mreža (slika 1).



Slika 1: Primer prostorije problema za određen ulaz

Da bismo pojednostavili rešavanje problema, mi njega svodimo na dobro poznati problem određivanja *poligona vidljivosti*, nakon čega se konačna površina senke dobija kao razlika površine prostorije (uključujući prepreke) i površine dobijenog poligona vidljivosti. U računarskoj geometriji, *poligon vidljivosti* tačke p u ravni sa preprekama predstavlja poligonalni region svih tačaka u ravni vidljivih iz tačke p. U literaturi postoje razna rešenja za određivanje *poligona vidljivosti*, a mi smo u ovom radu predstavili rešenje zasnovano na konceptu brišuće prave.

2 Algoritam

U ovoj sekciji prvo uvodimo naivni algoritam za konstrukciju *poligona vidljivosti*, nakon čega se predstavljamo njegovu optimizovanu verziju korišćenjem efikasnijih struktura podataka.

2.1 Naivni algoritam

Najosnovnija "teoretska" verzija ovog algoritma se zasniva na puštanju zraka iz tačke svetlosti pod svim uglovima i pronalasku tačaka poligona kao prvih preseka svakog zraka sa datim preprekama ili granicama prostorije. U praksi, budući da postoji beskonačan skup uglova koje je potrebno ispitati, korektna implementacija ovog algoritma nije izvodljiva.

Korektnu verziju naivnog algoritma možemo dobiti opažanjem da je dovoljno pustiti zrake samo u tačke samih prepreka, jer će sve promene u *poligonu vidljivosti* nastati u njima. Uzevši prethodno u obzir, dobijamo algoritam vremenske

složenosti $O(n^2)$, jer za svaku od n tačaka puštamo zrak čiji kraj pronalazimo izračunavanjem njegovog preseka sa svakom od $O(n)$ prepreka i biranjem najbližeg. Treba napomenuti da ovakva implementacija nije korektna za sve prostorije, jer može da se desi da je potrebno dodati više tačaka poligonu pri obradi jedne tačke prepreke, što u ovoj implementaciji nije moguće. Popravku ovoga, bar za specijalan slučaj problema prikazanog u ovom radu, možemo implementirati proverom da li najbliži presek predstavlja početnu ili krajnju tačku prepreke, zasnovano na ugaonoj obradi tačaka, i u tom slučaju dodavanjem prvog sledećeg preseka takođe u poligon.

2.2 Algoritam brišuće prave

Efikasniji algoritam dobijamo korišćenjem tehnike brišuće prave. Naime, možemo definisati polupravu sa početkom u tačkastom izvoru svetlosti i rotirati je za 360°, zaustavljajući se u svakoj tački prepreka (*tačkama događaja*) i u njima određujući najbližu vidljivu tačku (ili dve). Spajanjem ovih tačaka dobijamo traženi *poligon vidljivosti*.

Ključnu razliku u ovom algoritmu čini način obilaska *tačaka događaja*, kao i provera najbliže vidljive tačke u njima. Najpre sortiramo sve tačke prepreka prema uglu koji formiraju sa izvorom svetlosti, koristeći njihovu udaljenost od njega u slučaju jednakih uglova. Zatim formiramo sortirani skup svih ivica prepreka koje imaju presek sa početnom brišućom pravom (čiji izbor nije bitan, sve dok je rotiramo za punih 360° tokom algoritma), poznat kao *status*. Ivice u *statusu* su poređane tako da je na njegovom početku uvek ivica najbliža izvoru svetlosti, određena na osnovu njihove orijentacije i udaljenosti od izvora.

Zatim rotiramo brišuću pravu i zaustavljamo se u prvoj sledećoj *tački događaja*. Ako ona predstavlja početnu tačku neke ivice prepreke, tu ivicu dodajemo u *status*; ako predstavlja krajnju tačku, izbacujemo je iz *statusa*. Nakon toga, u logaritamskom vremenu dobijamo najbližu tačku vidljivosti u toj *tački događaja* kao presek ivice na početku *statusa* sa brišućom pravom. U slučaju da je najbliža ivica različita od prethodne najbliže, moramo prethodno dodati i presek sa prethodnom najbližom ivicom kako bismo održali ispravnost algoritma.

U odnosu na naivni algoritam, algoritam brišuće prave ima složenost $O(n \log n)$, jer u svakoj od n *tačaka događaja* u vremenskoj složenosti $O(\log n)$ pronalazimo sledeću tačku u *poligonu vidljivosti*.

3 Implementacija algoritma

U ovoj sekciji ćemo prikazati delove konkretne implementacije datog algoritma brišuće prave, sa osvrtom na kompleksnije i bitnije delove implementacije. Algoritam je implementiran u jeziku C++ korišćenjem Qt biblioteke, uz upotrebu klasa kao što su `QLineF`, `QPointF` i drugih komponenti za rad sa geometrijom i grafikom.

```
struct poredjenjeIvicaStatusaSA {
    const QPointF _izvorSvetlosti;
    const QLineF* _brisucaPrava;

    poredjenjeIvicaStatusaSA(const QPointF& izvorSvetlosti,
        const QLineF& brisucaPrava)
```

```
        : _izvorSvetlosti(izvorSvetlosti), _brisucaPrava(&brisucaPrava) {}

    bool operator()(const QLineF& ivica1, const QLineF& ivica2) const {
        if (ivica1.p1() == ivica2.p1() && ivica1.p2() == ivica2.p2()) {
            return false;
        }

        if (ivica1.p1() == ivica2.p2()) {
            return ShadowArea::orijentacija(ivica1.p2(), ivica1.p1(), ivica2.p1()) == 1;
        }
        if (ivica1.p1() == ivica2.p1()) {
            return ShadowArea::orijentacija(ivica1.p1(), ivica1.p2(), ivica2.p2()) == -1;
        }
        if (ivica1.p2() == ivica2.p1()) {
            return ShadowArea::orijentacija(ivica1.p1(), ivica1.p2(), ivica2.p2()) == 1;
        }
        if (ivica1.p2() == ivica2.p2()) {
            return ShadowArea::orijentacija(ivica1.p1(), ivica1.p2(), ivica2.p1()) == -1;
        }

        QPointF presek1, presek2;
        _brisucaPrava->intersects(ivica1, &presek1);
        _brisucaPrava->intersects(ivica2, &presek2);
        return QLineF(_izvorSvetlosti, presek1).length() < QLineF(_izvorSvetlosti, presek2).length();
    }
};
```

Listing 1: Implementacija komparatora ivica

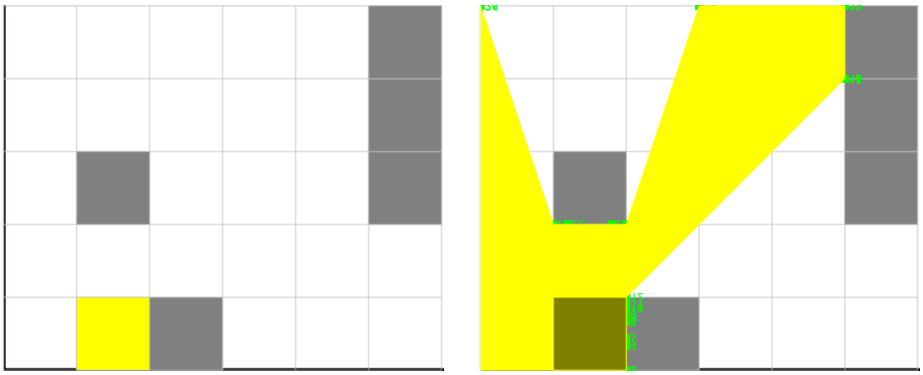
```
void ShadowArea::pokreniAlgoritam() {
    _brisucaPrava = QLineF(_izvorSvetlosti, QPointF(_izvorSvetlosti.x() - 1000, _izvorSvetlosti.y()));
    std::set<tackaDogadjajaSA> dogadjaji;
    std::set<QLineF, poredjenjeIvicaStatusaSA> status{{_izvorSvetlosti, _brisucaPrava}};

    for (auto &ivica : _ivicePrepreka) {
        QPointF p1 = ivica.p1();
        QPointF p2 = ivica.p2();

        double ugao1 = izracunajUgao(p1);
        double ugao2 = izracunajUgao(p2);

        double udaljenostOdSvetla1 = QLineF(_izvorSvetlosti, p1).length();
        double udaljenostOdSvetla2 = QLineF(_izvorSvetlosti,
```

```
            p2).length();

        dogadjaji.insert({p1, ugao1, udaljenostOdSvetla1});
        dogadjaji.insert({p2, ugao2, udaljenostOdSvetla2});

        QPointF presek;
        if (_brisucaPrava.intersects(ivica, &presek) ==
            QLineF::BoundedIntersection) {
            status.insert(ivica);
        }
    }

    QLineF prethodnaNajbliza = *status.begin();
    for (auto &dogadjaj : dogadjaji) {
        _brisucaPrava = QLineF(_izvorSvetlosti, dogadjaj.
            tacka + 10 * (dogadjaj.tacka - _izvorSvetlosti));
        for (auto &ivica : _endIvice[dogadjaj.tacka]) {
            status.erase(ivica);
        }
        for (auto &ivica : _startIvice[dogadjaj.tacka]) {
            status.insert(ivica);
        }
        if (!status.empty()) {
            QLineF najblizaIvica = *status.begin();
            if (najblizaIvica != prethodnaNajbliza) {
                QPointF presek;
                if (_brisucaPrava.intersects(
                    prethodnaNajbliza, &presek) == QLineF::
                    BoundedIntersection) {
                    _vidljiveTacke.push_back(presek);
                }
            }
            QPointF presek;
            if (_brisucaPrava.intersects(najblizaIvica, &
                presek) == QLineF::BoundedIntersection) {
                _vidljiveTacke.push_back(presek);
            }
            prethodnaNajbliza = najblizaIvica;
            AlgoritamBaza_updateCanvasAndBlock();
        }
    }
    _zavrsenAlgoritam = true;
    AlgoritamBaza_updateCanvasAndBlock();
}
```

Listing 2: Implementacija algoritma brišuće prave

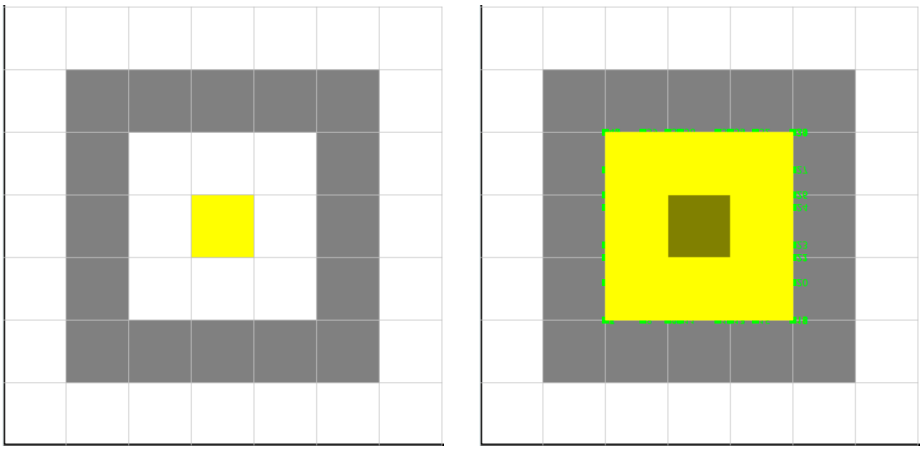
4 Vizuelizacija rezultata

U ovoj sekciji prikazaćemo rezultate algoritma na nekoliko primera, prikazivanjem rezultujućeg *poligona svetlosti* za različite ulazne konfiguracije prepreka, kao i odgovarajućih površina senke.



(a) Početna konfiguracija. (b) Rezultat algoritma.

Slika 2: Primer prostorije izračunate površine senke 13.5.



(a) Početna konfiguracija. (b) Rezultat algoritma.

Slika 3: Primer prostorije izračunate površine senke 24.